

Component Contracts

System Interface Specification

Defines inputs, outputs, and error behaviour for every significant component.

Table of Contents

1	OCR Service
2	Document Validator (Agent 1)
3	Document Parser (Agent 2)
4	Policy Checker (Agent 3)
5	Fraud Detector (Agent 4)
6	Decision Engine (Agent 5)
7	Trace Builder (Agent 6)
8	Database Layer
9	REST API Endpoints

FILE · services/ocr.py

```
extract_text_from_image(file_path: str) → str
```

Input

Parameter	Type	Description
file_path	str	Absolute path to a JPEG, PNG, or PDF file on disk

Output

Type	Description
str	Raw text extracted from the document. Layout is preserved where possible.

Error Behaviour

Condition	Behaviour
Missing OPENAI_API_KEY	Returns "ERROR: API KEY MISSING"
PDF extraction fails	Returns "" (empty string), logs error
Image Vision API fails	Returns "" (empty string), logs error
Unsupported file format	Falls through to Vision API path (no explicit error)

FILE · agents/document_validator.py CLASS · DocumentValidator

```
validate(claim_category: str, document_texts: List[str]) → Dict[str, Any]
```

Input

Parameter	Type	Description
claim_category	str	One of: CONSULTATION, DIAGNOSTIC, PHARMACY, DENTAL, VISION, ALTERNATIVE_MEDICINE
document_texts	List[str]	Raw text content of each uploaded document (from OCR)

Output — Success

```
{ "status": "SUCCESS",
  "message": "All required documents are present.",
  "details": {
    "classified_docs": [
      {"doc_type": "PRESCRIPTION", "confidence": 0.95},
      {"doc_type": "HOSPITAL_BILL", "confidence": 0.92}
    ],
    "optional_found": ["LAB_REPORT"]
  }
}
```

Output — Failure

```
{ "status": "FAILED",
  "error_type": "MISSING_REQUIRED_DOCUMENTS",
  "message": "Your Consultation claim is incomplete: missing Hospital Bill.",
  "details": {
    "classified_docs": [...],
    "missing": ["HOSPITAL_BILL"],
    "optional_found": []
  }
}
```

Error Behaviour

Condition	Behaviour
Invalid claim_category	Returns {"status": "FAILED", "message": "Invalid claim category: ..."}
Empty/short document text	Classified as {"doc_type": "UNKNOWN", "confidence": 0.0}
OpenAI API failure	Document classified as UNKNOWN with confidence 0.0

FILE · agents/document_parser.py CLASS · DocumentParser

```
parse(document_texts: List[str], classified_docs: List[Dict]) → List[Dict[str, Any]]
```

Input

Parameter	Type	Description
document_texts	List[str]	Raw text content of each document
classified_docs	List[Dict]	Output from Agent 1's details.classified_docs. Each dict has doc_type and confidence.

Output — Example

```
[{ "doc_type": "PRESCRIPTION",
  "classification_confidence": 0.95,
  "extracted_data": {
    "patient_name": "Rajesh Kumar",
    "date": "2024-11-01",
    "hospital_or_clinic_name": "City Clinic, Bengaluru",
    "doctor_name": "Dr. Arun Sharma",
    "doctor_registration": "KA/45678/2015",
    "diagnosis": "Viral Fever",
    "medicines": [{"name": "Paracetamol", "dosage": "650mg", "frequency": "TID",
    "duration": "5 days"}],
    "tests_ordered": ["CBC"]
  }
}]
```

Type-Specific Fields

Document Type	Additional Fields
PRESCRIPTION	doctor_name, doctor_registration, diagnosis, medicines[], tests_ordered[]
HOSPITAL_BILL	bill_number, line_items[] (description, amount, quantity), total_amount, tax_amount
PHARMACY_BILL	pharmacy_name, medicines[] (name, batch_number, expiry_date, quantity, amount), total_amount
LAB_REPORT	lab_name, test_name, results[] (parameter, value, unit, normal_range)

Error Behaviour

Condition	Behaviour
doc_type is UNKNOWN	Returns {"doc_type":"UNKNOWN","extracted_data":{},"status":"SKIPPED","reason":"Unrecognized or empty document"}

Condition	Behaviour
OpenAI extraction fails	Returns {"error": "Extraction failed", "details": "..."} inside extracted_data

FILE · agents/policy_checker.py CLASS · PolicyChecker

```
check(claim_data: Dict, extracted_docs: List[Dict]) → Dict[str, Any]
```

Input

Parameter	Type	Description
claim_data	Dict	{ "member_id": "EMP001", "claim_category": "CONSULTATION", "claimed_amount": 1500.0, "treatment_date": "2024-11-01" }
extracted_docs	List[Dict]	Output from Agent 2

Output — All Checks Pass

```
{  
  "status": "SUCCESS",  
  "policy_checks": [  
    {"rule": "WAITING_PERIOD", "status": "PASS", "message": "All waiting periods  
cleared."},  
    {"rule": "EXCLUSION_CHECK", "status": "PASS", "message": "No exclusions triggered."},  
    {"rule": "PRE_AUTH_CHECK", "status": "PASS", "message": "Pre-auth not required."},  
    {"rule": "FINANCIAL_CALCULATION", "status": "PASS",  
      "calculations": {  
        "claimed_amount": 1500.0, "network_discount": 0.0,  
        "after_discount": 1500.0, "copay_deducted": 150.0,  
        "after_copay": 1350.0, "sub_limit": 2000,  
        "exceeds_sublimit": false, "final_approved_amount": 1350.0  
      }}  
  ]  
}
```

Financial Calculation Order

Step	Operation
1	Network discount applied first (if network hospital)
2	Copay deducted from post-discount amount
3	Sub-limit cap applied last

Error Behaviour

Condition	Behaviour
Unknown category	Returns {"status": "FAILED", "reason": "Category X not covered by policy."}

Condition	Behaviour
Unknown member_id	FAIL on waiting period check: "Member ID [X] not found in our records."
Unparseable treatment date	Raises ValueError (caught by pipeline try/except)

FILE · agents/fraud_detector.py CLASS · FraudDetector

```
detect(claim_data: Dict, extracted_docs: List[Dict], history: List[Dict]) → Dict[str, Any]
```

Input

Parameter	Type	Description
claim_data	Dict	Same structure as Agent 3 input
extracted_docs	List[Dict]	Output from Agent 2
history	List[Dict]	Past claims: {"claim_id", "member_id", "date", "amount", "verdict"}

Output

```
{ "fraud_score": 0.4,
  "flags": ["HIGH_FREQUENCY: 2 claims already submitted for 2024-10-30."],
  "manual_review_recommended": false
}
```

Fraud Score Contribution

Signal	Score Added
Same-day frequency >= limit (default 2)	+0.4
Duplicate claim (same date + amount)	+0.6
High value > Rs 25,000	+0.3
Identity mismatch across documents	+0.7

Score is capped at **1.0**. **manual_review_recommended** is **true** when score >= **0.8**.

Error Behaviour

Condition	Behaviour
history is None	Treated as empty list []
Missing patient_name in docs	Name check skipped (no flag raised)

FILE · agents/decision_engine.py CLASS · DecisionEngine

```
decide(policy_res: Dict, fraud_res: Dict) → Dict[str, Any]
```

Input

Parameter	Type	Description
policy_res	Dict	Output from Agent 3
fraud_res	Dict	Output from Agent 4

Output

```
{ "verdict": "APPROVED",  
  "reason": "All checks passed.",  
  "approved_amount": 1350.0,  
  "confidence_score": 0.95  
}
```

Verdict Logic (priority order)

Condition	Verdict
fraud_score >= 0.8 OR manual_review_recommended	MANUAL_REVIEW
Any policy check has status FAIL	REJECTED
final_approved_amount < claimed_amount	PARTIAL
All checks pass	APPROVED

Confidence score formula: **1.0 - (fraud_score × 0.5)** — ranges from 0.5 (max fraud) to 1.0 (no fraud signals).

FILE · agents/trace_builder.py CLASS · TraceBuilder

```
build_trace(claim_id, member_id, category, val_res, parsing_res, policy_res, fraud_res,
decision_res) → Dict[str, Any]
```

Input Parameters

Parameter	Type	Description
claim_id	str	Generated claim ID (e.g. "CLM-A3F2B1C4")
member_id	str	Member identifier
category	str	Claim category
val_res	Dict	Output from Agent 1
parsing_res	List[Dict]	Output from Agent 2
policy_res	Dict	Output from Agent 3
fraud_res	Dict	Output from Agent 4
decision_res	Dict	Output from Agent 5

Output

```
{ "claim_id": "CLM-A3F2B1C4",
  "timestamp": "2024-11-01T10:30:00.000000",
  "summary": {
    "member_id": "EMP001", "category": "CONSULTATION",
    "verdict": "APPROVED", "approved_amount": 1350.0, "confidence": 0.95
  },
  "stages": [
    {"stage": 1, "name": "Document Validation", "status": "SUCCESS", "details": [...]},
    {"stage": 2, "name": "Document Parsing", "status": "SUCCESS", "details": [...]},
    {"stage": 3, "name": "Policy Checking", "status": "SUCCESS", "details": [...]},
    {"stage": 4, "name": "Fraud Detection", "status": "SUCCESS", "details": {...}},
    {"stage": 5, "name": "Decision Engine", "status": "SUCCESS", "details": {...}}
  ]
}
```

This trace is persisted in the database and is viewable from the frontend dashboard.

FILE · database/db.py

init_db() → None

Creates the **claim_history** table if it does not exist. Prints a warning on failure.

save_claim(claim_data: Dict, trace_data: Dict) → None

Parameter	Type	Fields Required
claim_data	Dict	claim_id, member_id, date, amount, verdict
trace_data	Dict	Full trace JSON from Agent 6

get_history(member_id: str) → List[Dict]

Parameter	Type	Description
member_id	str	If empty string, returns all claims. Otherwise filters by member.

Returns: List of dicts with **claim_id**, **member_id**, **date**, **amount**, **verdict**, **created_at**, **trace**.

POST · /submit-claim — Content-Type: multipart/form-data

Field	Type	Required
member_id	string	Yes
claim_category	string	Yes
claimed_amount	float	Yes
treatment_date	string (YYYY-MM-DD)	Yes
files	File[]	Yes (1 or more)

Response:

```
{"status": "ACCEPTED", "job_id": "uuid"}
```

GET · /claim-status/{job_id}**Response while running:**

```
{"status": "RUNNING", "current_step": 2, "result": null}
```

Response on completion:

```
{"status": "SUCCESS", "current_step": null, "result": {"claim_id": "...", "verdict": "...", "trace": {...}}}
```

Response on failure:

```
{"status": "FAILED", "result": {"status": "VALIDATION_FAILED", "failed_step": 0, "error_message": "..."} }
```

GET · /history**Response:** Array of claim history objects with full trace data.

Another engineer could reimplement any component using only this specification.